

LLM 中的 Tokenizers

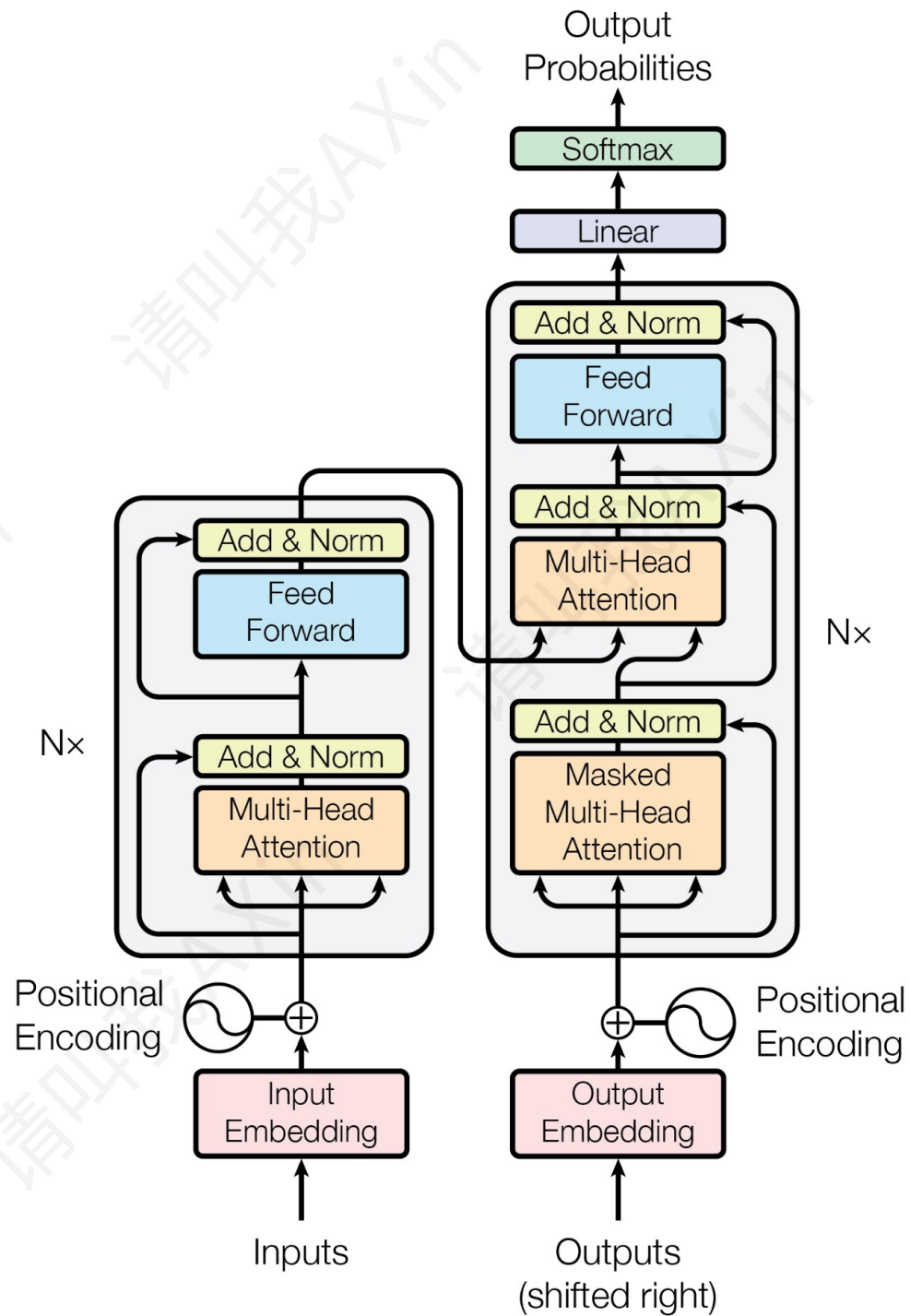
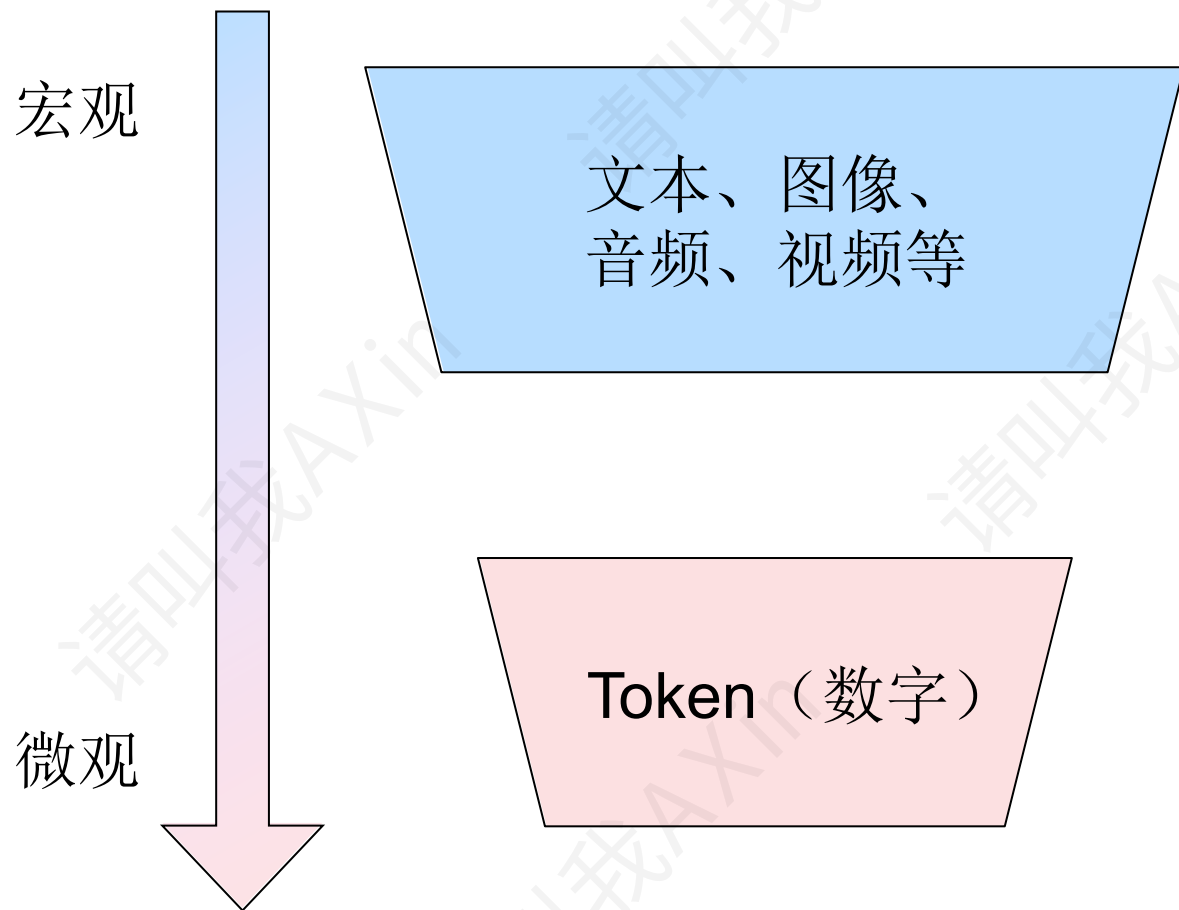
请叫我AXin

2023 年 12 月 22 日

Outline

- Transformer 的输入是什么
- Tokenizer 的作用
- 三种不同分词粒度的 Tokenizers
- 四种常用的 Subword Tokenizers

Transformer 的输入是什么？



Transformer 的输入是什么？

"This is a input text."

Tokenization

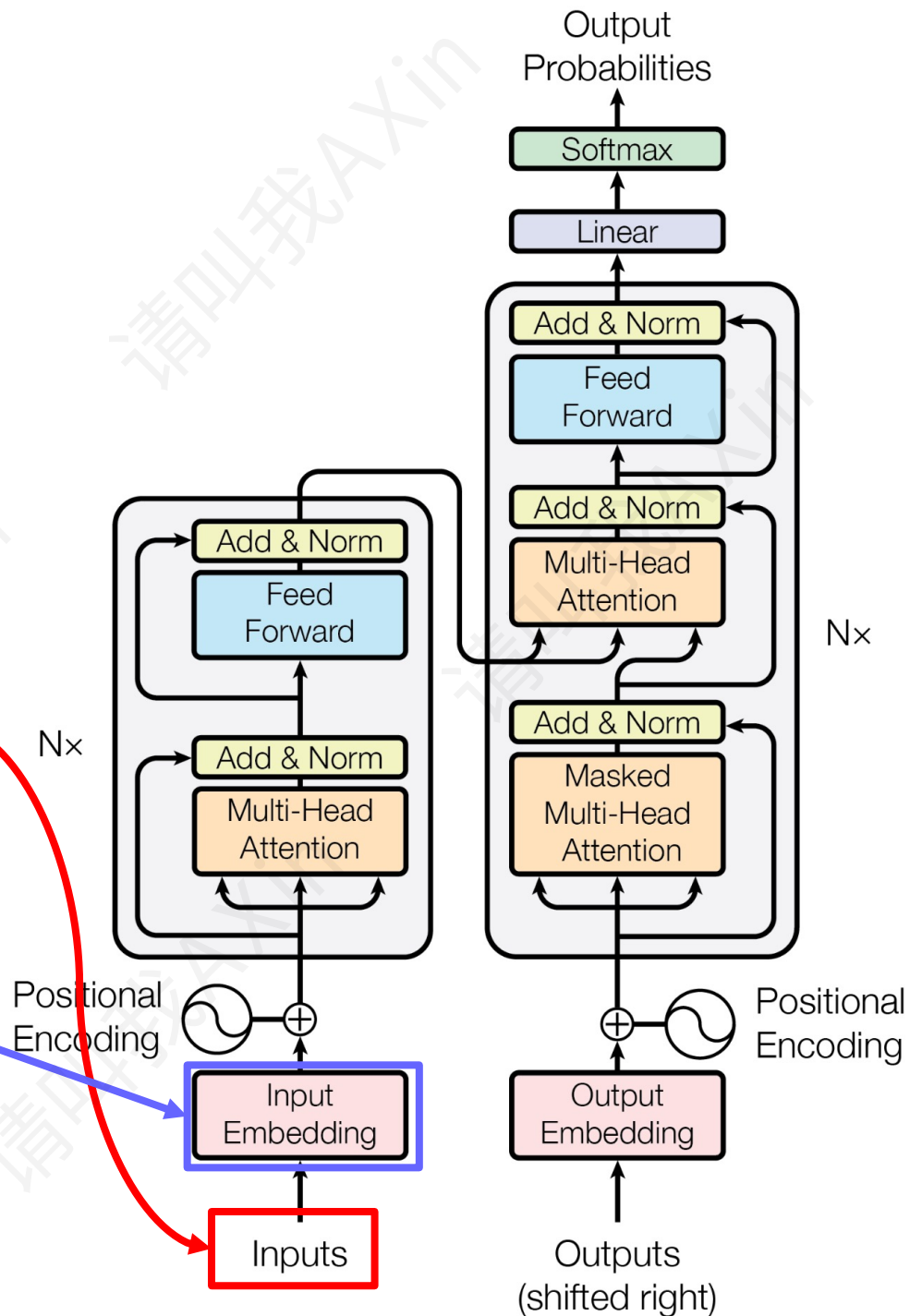
[CLS]	This	is	a	input	.	[SEP]
101	2023	2003	1037	7953	1012	102

每个token都有一个id

Embeddings

0.0390,	-0.0558,	-0.0440,	0.0119,	0069,	0.0199,	-0.0788,
-0.0123,	0.0151,	-0.0236,	-0.0037,	0.0057,	-0.0095,	0.0202,
-0.0208,	0.0031,	-0.0283,	-0.0402,	-0.0016,	-0.0099,	-0.0352,
...	...	...	...	...	...	...

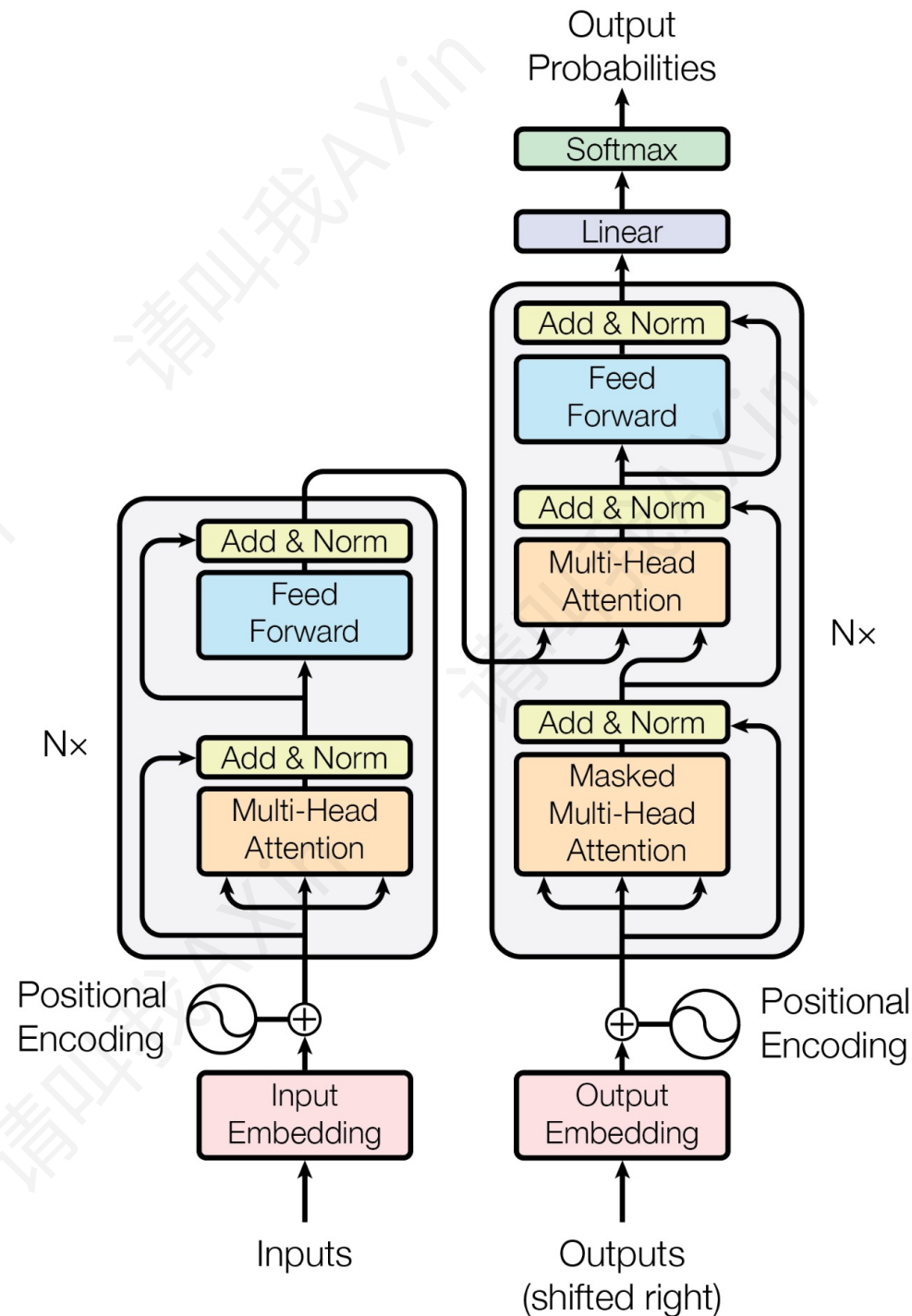
Token对应的embedding是学习得到的



Transformer 的输入是什么？

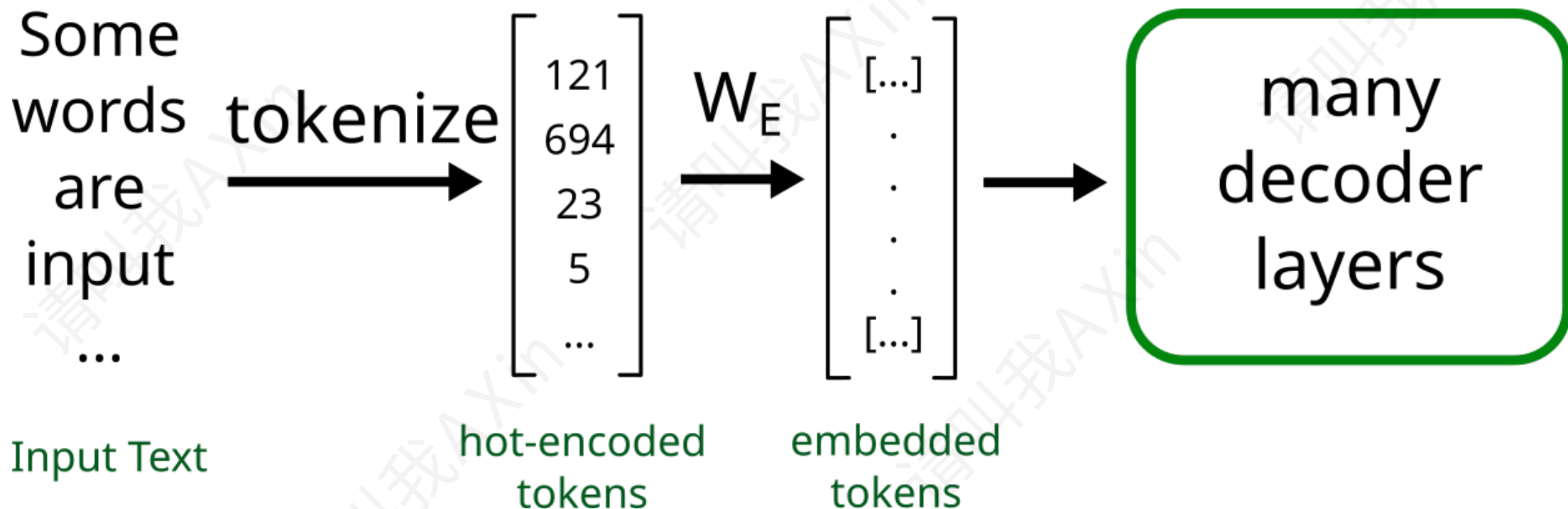
- Embedding layer 是 Transformer 模型的一部分

```
class InputEmbedding(nn.Module):  
    """Implementation of the input embeddings"""  
  
    def __init__(self, d_model: int, vocab_size: int) -> None:  
        super().__init__()  
        self.d_model = d_model  
        self.vocab_size = vocab_size  
        self.embedding = nn.Embedding(vocab_size, d_model)  
  
    def forward(self, x):  
        return self.embedding(x) * math.sqrt(self.d_model)
```



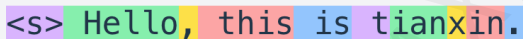
Tokenizer 的作用

- 将文本序列转化为数字序列 (token编号) , 作为 transformer的输入
- 是训练和微调 LLM 必不可少的一部分



Tokenizer 的作用

- 将文本序列转化为数字序列（token编号），作为 transformer的输入

gpt-4 / gpt-3.5-turbo / text-embedding-ada-002 ▾		LLaMA / Llama 2 ▾	
Hello, this is tianxin.		Hello, this is tianxin.	
TOKENS 8		TOKENS 10	
CHARACTERS 23		CHARACTERS 23	
			
[9906, 11, 420, 374, 259, 1122, 59866, 13]		[1, 15043, 29892, 445, 338, 260, 713, 29916, 262, 29889]	

HuggingFace: <https://huggingface.co/spaces/Xenova/the-tokenizer-playground>

OpenAI: <https://platform.openai.com/tokenizer>

三种不同分词粒度的 Tokenizers

- Word-based
- Character-based
- Subword-based

Word-based Tokenizers

https://huggingface.co/docs/transformers/tokenizer_summary

■ 将文本划分为一个个词（包括标点）

"Don't you love 🤗 Transformers? We sure do."

↓ 没有把标点分开

["Don't", "you", "love", "🤗", "Transformers?", "We", "sure", "do."]

↓ 把标点也分开

["Don", "'", "t", "you", "love", "🤗", "Transformers", "?", "We", "sure", "do", "."]



Don't 其实是 Do not, 应该划分为 Do 和 n't

引入规则的后果：事情开始变得复杂起来了！

spaCy和Moses: ["Do", "n't", "you", "love", "🤗", "Transformers", "?", "We", "sure", "do", "."]

Word-based Tokenizers

https://huggingface.co/docs/transformers/tokenizer_summary

■ 将文本划分为一个个词（包括标点）

➤ 优点：符合人的自然语言和直觉

the	→	1
of	→	2
and	→	3
to	→	4
in	→	5
was	→	6
the	→	7
is	→	8
for	→	9
as	→	10
on	→	11
with	→	12
that	→	13
dog	→	14
dogs	→	15

➤ 缺点：

相同意思的词被划分为不同的token

the	→	1
of	→	2
and	→	3
to	→	4
in	→	5
was	→	6
the	→	7
is	→	8
for	→	9
as	→	10
on	→	11
with	→	12
that	→	13
...	→	...
malapropism	→	170,000

词表非常大

Word-based Tokenizers

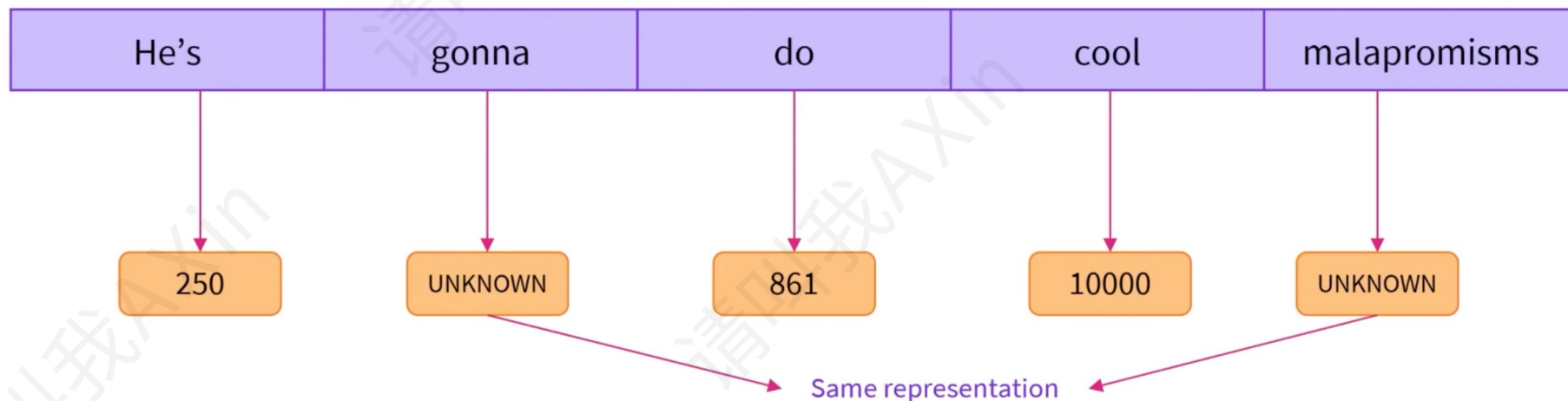
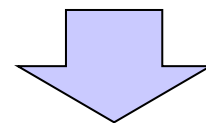
https://huggingface.co/docs/transformers/tokenizer_summary

■ 将文本划分为一个个词（包括标点）

the	→	1
of	→	2
and	→	3
to	→	4
in	→	5
was	→	6
the	→	7
is	→	8
for	→	9
as	→	10
on	→	11
with	→	12
that	→	13
...	→	...
malapropism	→	170,000

词表非常大

限制词表大小，比如上限为10,000，未知的词用特殊的token表示



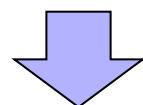
信息丢失！模型性能大打折扣！

Character-based Tokenizers

https://huggingface.co/docs/transformers/tokenizer_summary

- 将文本划分为一个个字符（包括标点）

"What is your name?"



["W", "h", "a", "t", " ", "i", "s", " ", "y", "o", "u", "r", " ", "n", "a", "m", "e", "?"]

a	→	1	the	→	1
b	→	2	of	→	2
c	→	3	and	→	3
d	→	4	to	→	4
e	→	5	in	→	5
f	→	6	was	→	6
g	→	7	the	→	7
...	→	...	is	→	8
1	→	27	for	→	9
2	→	28	as	→	10
3	→	29	on	→	11
...	→	...	with	→	12
!	→	37	that	→	13
...	→	...	...	→	...
à	→	256	malapropism	→	170,000

Character-based
vocabulary

Word-based
vocabulary

- 优点:

- 可以表示任意（英文）文本，不会出现 word-based 中的 unknown 情况
- 对西文来说，不需要很大的词表，例如英文只需不到 256 个字符

Character-based Tokenizers

https://huggingface.co/docs/transformers/tokenizer_summary

- 将文本划分为一个个字符（包括标点）
- 缺点
 - 相对单词来说信息量非常低，模型性能一般很差
 - 相对于 word-based 来说，会产生很长的 Token 序列
 - 中文也需要很大一个词表

Word-based tokenization

250	861	10000	UNKNOWN
-----	-----	-------	---------

Character-based tokenization

12	5	20	40	19	4	15	3	15	15	12	13	1	12	15	16	18	15	16	9	19	13	19
----	---	----	----	----	---	----	---	----	----	----	----	---	----	----	----	----	----	----	---	----	----	----

Subword-based Tokenizers

https://huggingface.co/docs/transformers/tokenizer_summary

- 非常大的词表
- 会经常出现未知词
- 相同词的不同形式（**cat**和**cats**）会分为不同 **token**

Word-based Tokenizer

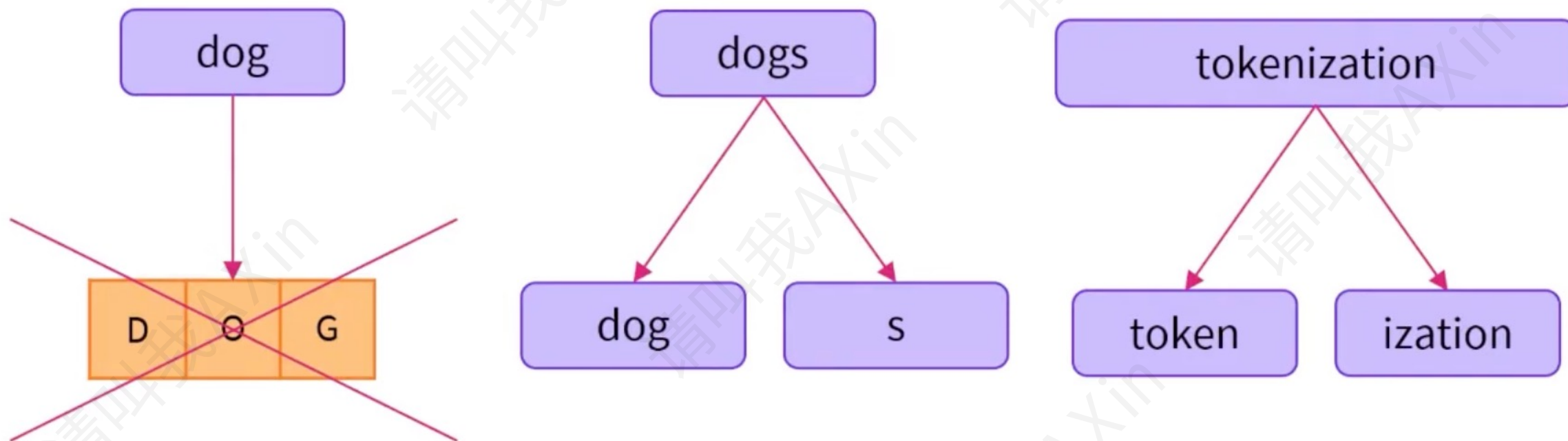
- **Token** 序列非常长
- 单个 **token** 信息量太少

Character-based Tokenizer

- 常用词不应该再被切分成更小的 **token** 或子词（**subword**）
- 不常用的词或词群应该用子词来表示

Subword-based Tokenizers

https://huggingface.co/docs/transformers/tokenizer_summary

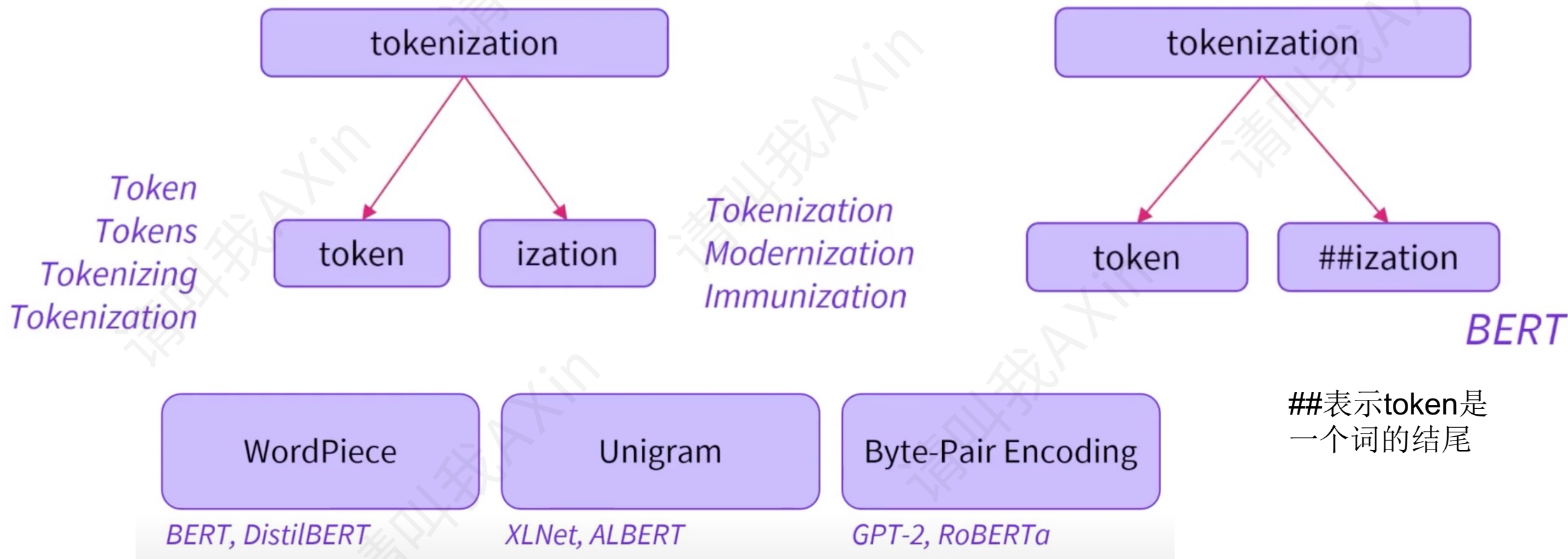


- 常用词不应该再被切分成更小的 **token** 或子词 (**subword**)
- 不常用的词或词群应该用子词来表示

Subword-based Tokenizers

https://huggingface.co/docs/transformers/tokenizer_summary

- 使用 subword 划分符合英文词群，能够充分表意



作用：将文本序列转化为数字（token）序列，作为transformer的输入

Tokenizer

分词粒度

Word

按照词进行分词，如英文“Today is Sunday.”则根据空格或标点进行分割[Today, is, Sunday, .]

优点：简单，符合人类用词方式

缺点：

每个word都分配一个ID，所需的Vocabulary根据语料大小而不同
会将意思一致的词分成两个完全不同的ID，如cat, cats。

Character

按照单字符进行分词，如英文“Today is Sunday.”则会分割成[T, o, d, a, y, i, s, S, u, n, d, a, y, .]

优点：英文Vocabulary相对小的多，适合中文（词表很大）

缺点：

对西文来说，分词后的每个字符是毫无意义的，只有组装后才有意义
输入的长度会变得很长，不适合西文

Subword

按照词的子词进行分词，如英文“Today is Sunday.”则会分割成[To, day, is, s, un, day, .]

优点：Vocabulary的大小固定，能保持语义，适合多语言

常见方法：BPE, Byte-Level BPE, WordPiece, SentencePiece, Unigram

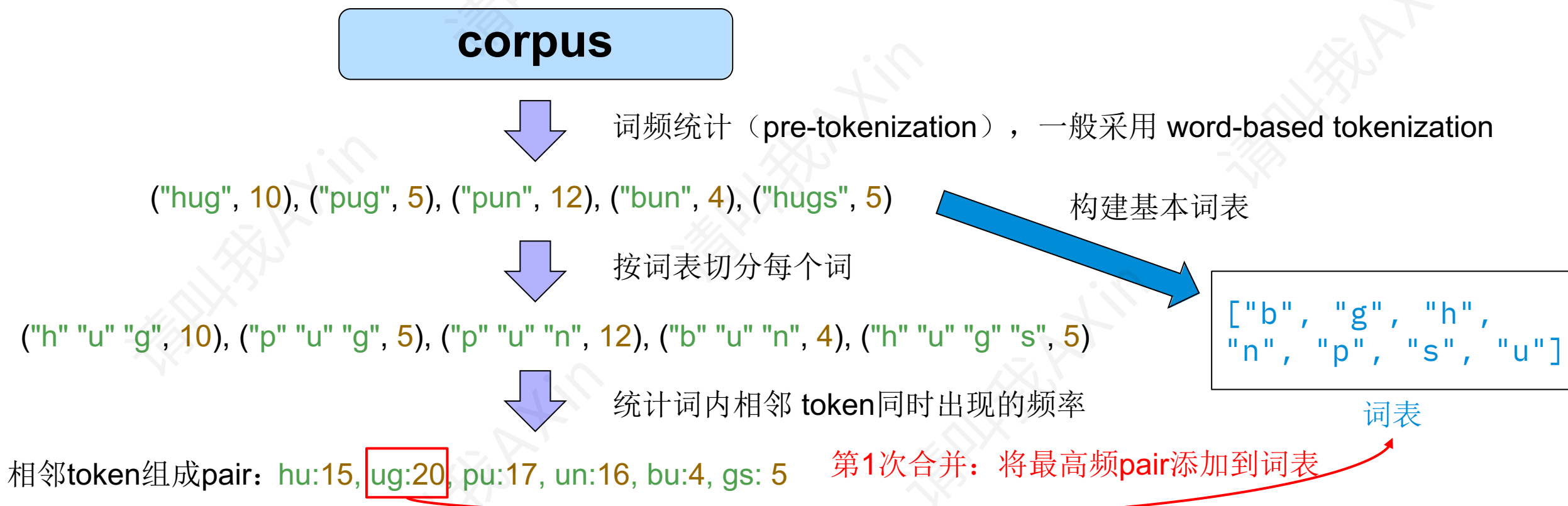
Subword 分词方法	典型模型
BPE/BBPE	GPT, GPT-2, GPT-J, GPT-Neo, RoBERTa, BART, LLaMA, ChatGLM-6B, Baichuan
WordPiece	BERT, DistilBERT, MobileBERT
Unigram	ALBERT, T5, mBART, XLNet

四种常用的 Subword Tokenizers

- Byte-Pair Encoding (BPE)
- WordPiece
- SentencePiece
- Unigram

Byte-Pair Encoding (BPE) Tokenization

- BPE 算法包含两部分：“词频统计” 与 “词表合并”

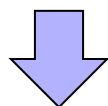


- Sennrich, Rico, Barry Haddow, and Alexandra Birch. "Neural machine translation of rare words with subword units." *arXiv preprint arXiv:1508.07909* (2015).
- https://huggingface.co/docs/transformers/tokenizer_summary

Byte-Pair Encoding (BPE) Tokenization

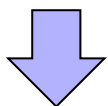
■ BPE 算法包含两部分：“词频统计” 与 “词表合并”

("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)



使用刚刚添加到词表的“ug”替换

("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)



统计词内相邻 token (pair) 共同出现的词频

hug:15, pug:5, pu:12, **un:16**, bu:4, ugs: 5

第2次合并：将最高频pair添加到词表

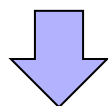
["b", "g", "h",
"n", "p", "s",
"u", "ug"]

词表

Byte-Pair Encoding (BPE) Tokenization

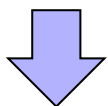
■ BPE 算法包含两部分：“词频统计” 与 “词表合并”

("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)



使用刚刚添加到词表的“un”替换

("h" "ug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("h" "ug" "s", 5)



统计相邻 token (pair) 共同出现的词频

hug:15, pug:5, pun:12, bun:4, ugs: 5

第3次合并：将最高频pair添加到词表

["b", "g", "h",
"n", "p", "s",
"u", "ug", "un"]

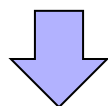
词表

- Sennrich, Rico, Barry Haddow, and Alexandra Birch. "Neural machine translation of rare words with subword units." *arXiv preprint arXiv:1508.07909* (2015).
- https://huggingface.co/docs/transformers/tokenizer_summary

Byte-Pair Encoding (BPE) Tokenization

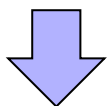
■ BPE 算法包含两部分：“词频统计” 与 “词表合并”

("h" "ug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("h" "ug" "s", 5)



使用刚刚添加到词表的“hug”替换

("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("hug" "s", 5)



停止合并，得到最终词表（BPE的合并次数是超参数）



使用词表进行 tokenization

bug => ["b", "ug"]
mug => ["<unk>", "ug"]

GPT的词汇表大小为40478，因为它有478个基本字符，并且在40000次合并后停止。

["b", "g", "h",
"n", "p", "s",
"u", "ug", "un",
"hug"]

词表

- Sennrich, Rico, Barry Haddow, and Alexandra Birch. "Neural machine translation of rare words with subword units." *arXiv preprint arXiv:1508.07909* (2015).
- https://huggingface.co/docs/transformers/tokenizer_summary

Byte-level BPE (BBPE)

■ BPE 的缺点

- 包含所有可能的基本字符 (token) 的基本词汇表可能相当大
- 例如, 将所有 Unicode 字符都被视为基本字符 (如中文)

■ 改进: Byte-level BPE

- 将字节 (byte) 视为基本 token
- 两个字节合并即可以表示 Unicode
 - ▣ 比如中文、日文、阿拉伯文、表情符号等等

GPT-2的词汇表大小为50257, 对应于256字节的基本token、一个特殊的文本结束token和通过50000个合并得到的token。

WordPiece Tokenization

<https://static.googleusercontent.com/media/research.google.com/ja//pubs/archive/37842.pdf>

■ 大体和BPE类似

- 构建基本词表时，除第一个字母，会添加##作为前缀（BERT）

word => [w, ##o, ##r, ##d]

- 使用类似联合概率的大小而不是次数对 token 进行合并，公式如下：

$$\text{pair得分} = \frac{\text{pair出现的次数}}{\text{token1出现的次数} \times \text{token2出现的次数}}$$

- 通过将 pair 的频率除以其单个 token 的频率的乘积，该算法优先考虑单个 token 在词表中不太频繁的 pair 进行合并

WordPiece Tokenization

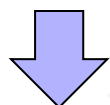
- 通过将 pair 的频率除以其单个 token 的频率的乘积，该算法优先考虑单个 token 在词表中不太频繁的 pair 进行合并

$$\text{pair得分} = \frac{\text{pair出现的次数}}{\text{token1出现的次数} \times \text{token2出现的次数}}$$

例如，它不一定会合并（“un”、“##able”），即使这对词在词汇表中出现得很频繁，因为“un”和“###able”这两对词可能都会出现在很多其他单词中，并且频率很高。相比之下，像（“hu”、“##gging”）这样的一对可能会更快地融合（假设“hugging”一词经常出现在词汇表中），因为“hu”和“##ggig”单独出现的频率可能较低。

WordPiece Tokenization

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)



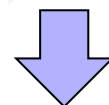
按词表切分

("h" "##u" "##g", 10), ("p" "##u" "##g", 5), ("p" "##u" "##n", 12), ("b" "##u" "##n", 4), ("h" "##u" "##g" "##s", 5)



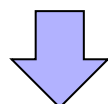
计算pair出现次数

hu: 15, ug: 20, pu: 17, un: 16, bu: 4, gs: 5



计算单个token出现次数

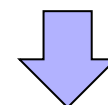
h: 15, u: 36, g: 20, p: 17, n: 16, b: 4, s: 5



计算每个pair的概率

hu: $15/(15 \cdot 36)$, ug: $20/(36 \cdot 20)$, pu: $17/(17 \cdot 36)$, un: $16/(36 \cdot 16)$, bu: $4/(4 \cdot 36)$, gs: $5/(20 \cdot 5)$

合并概率最大的pair



("##g", "##s") $\rightarrow$ ("##gs")

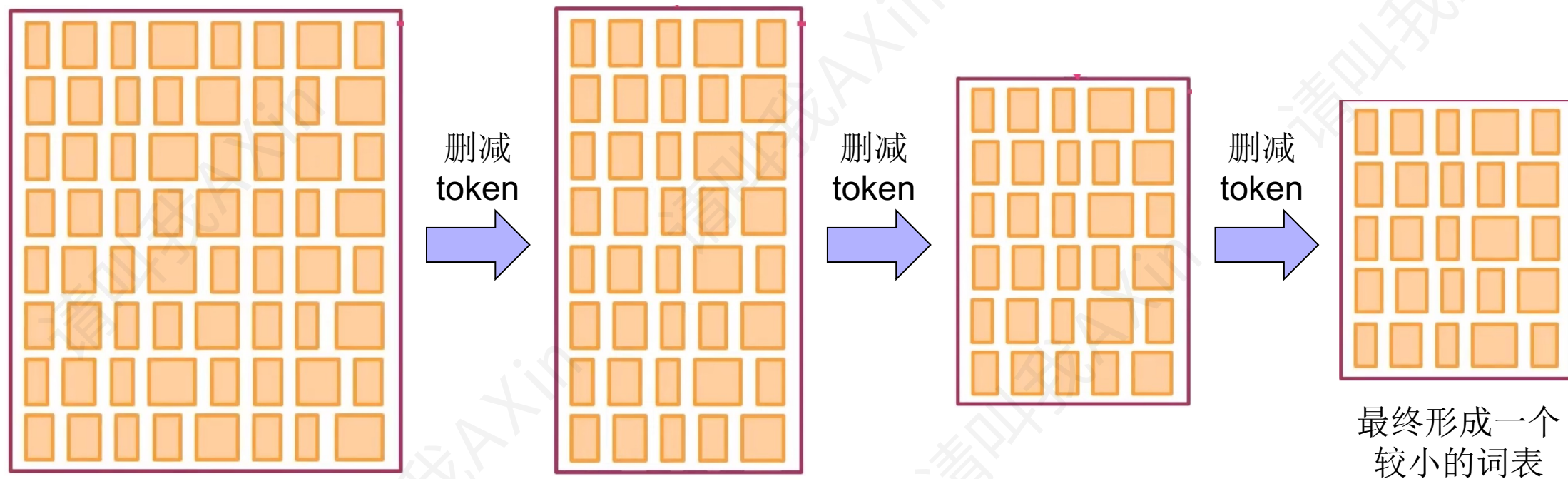
["b", "h", "p", "##g",
"##n", "##s", "##u"]

词表

Unigram Tokenization

<https://arxiv.org/pdf/1804.10959.pdf>

- Unigram 算法经常在 SentencePiece 中使用，是 ALBERT、T5、mBART、Big Bird 和 XLNet 等模型使用的 tokenization 算法

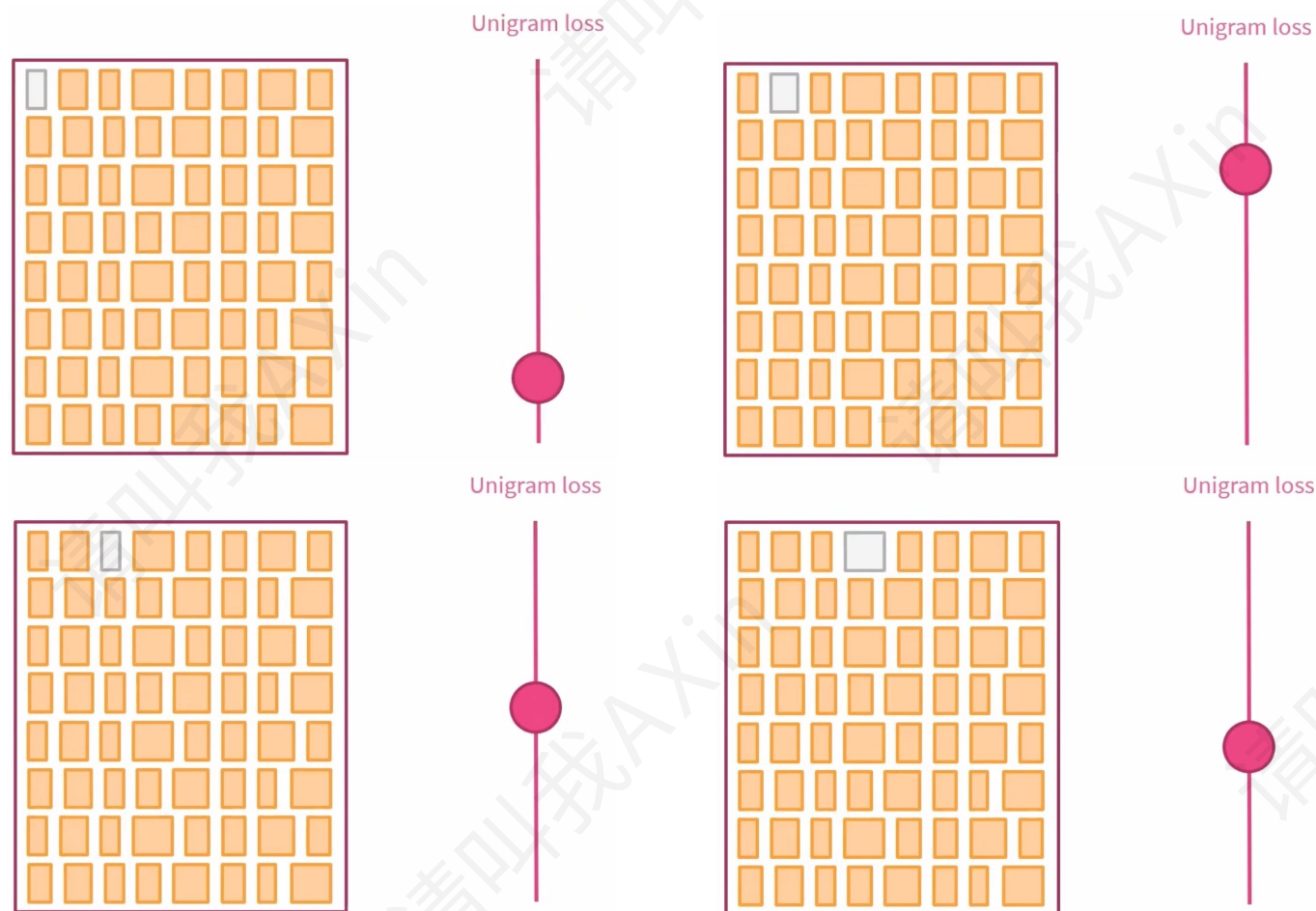


初始一个很大的词表

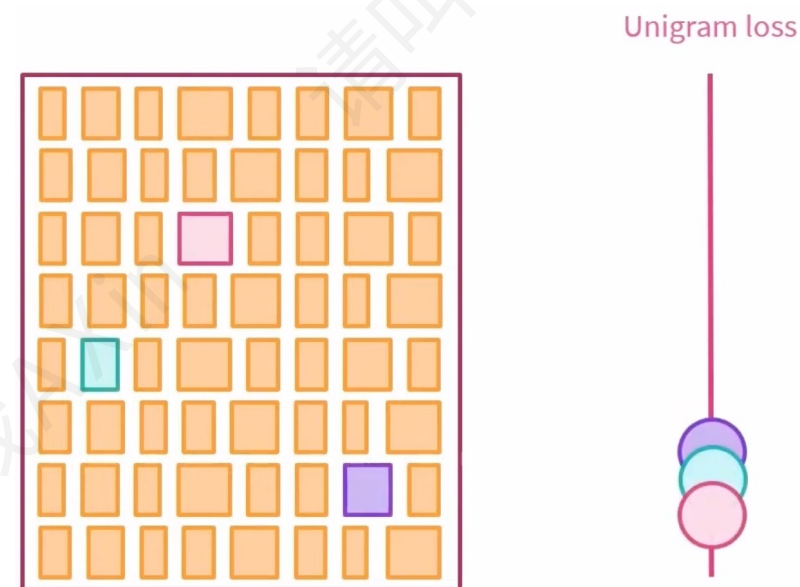
<https://huggingface.co/learn/nlp-course/chapter6/7>
<https://www.youtube.com/watch?v=TGZfZVuF9Yc>

Unigram Tokenization

■ 如何删减 token?

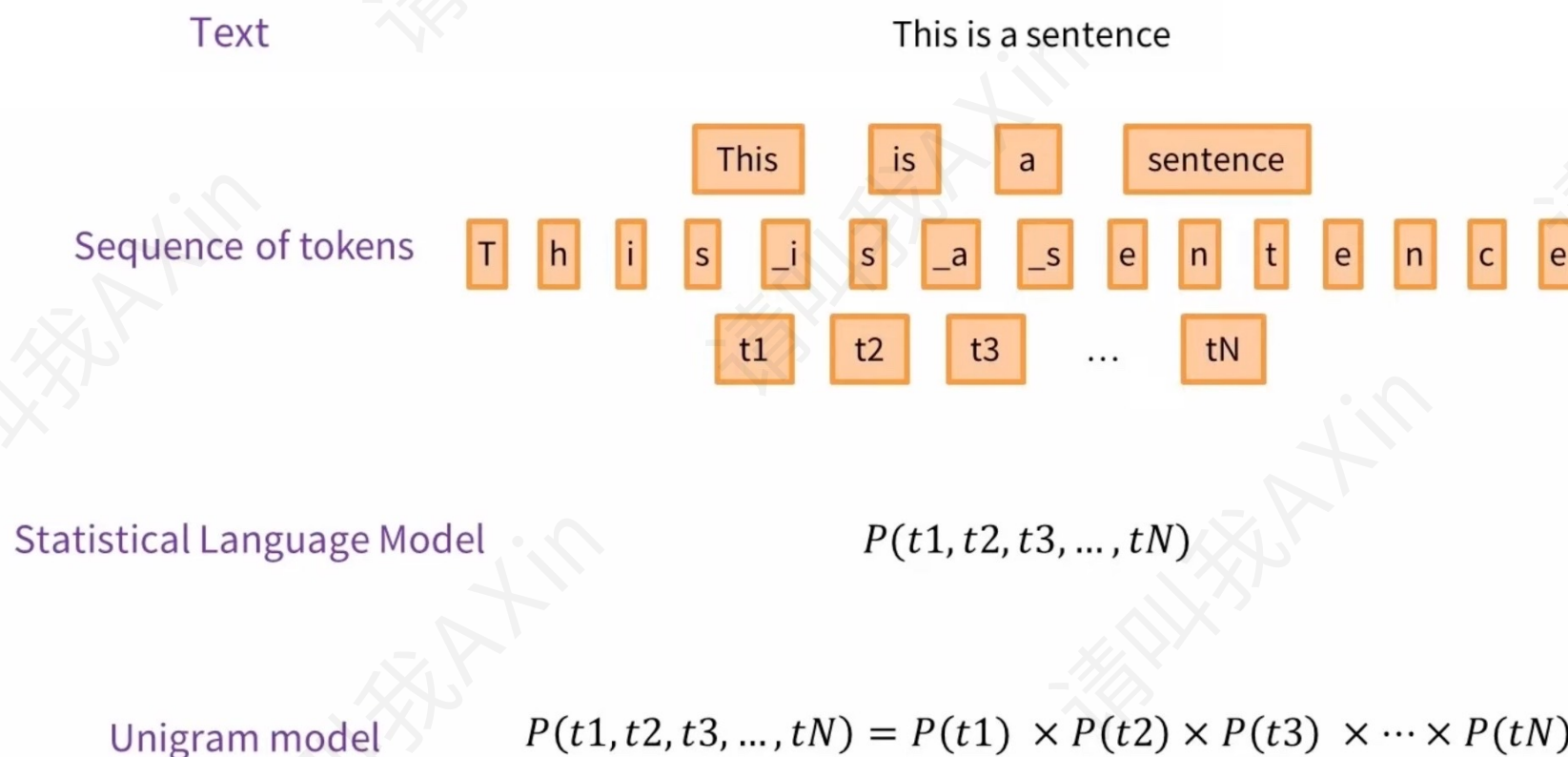


尝试删去一个 token，并计算对应的 unigram loss，删除 $p\%$ 使得 loss 增加最少的 token



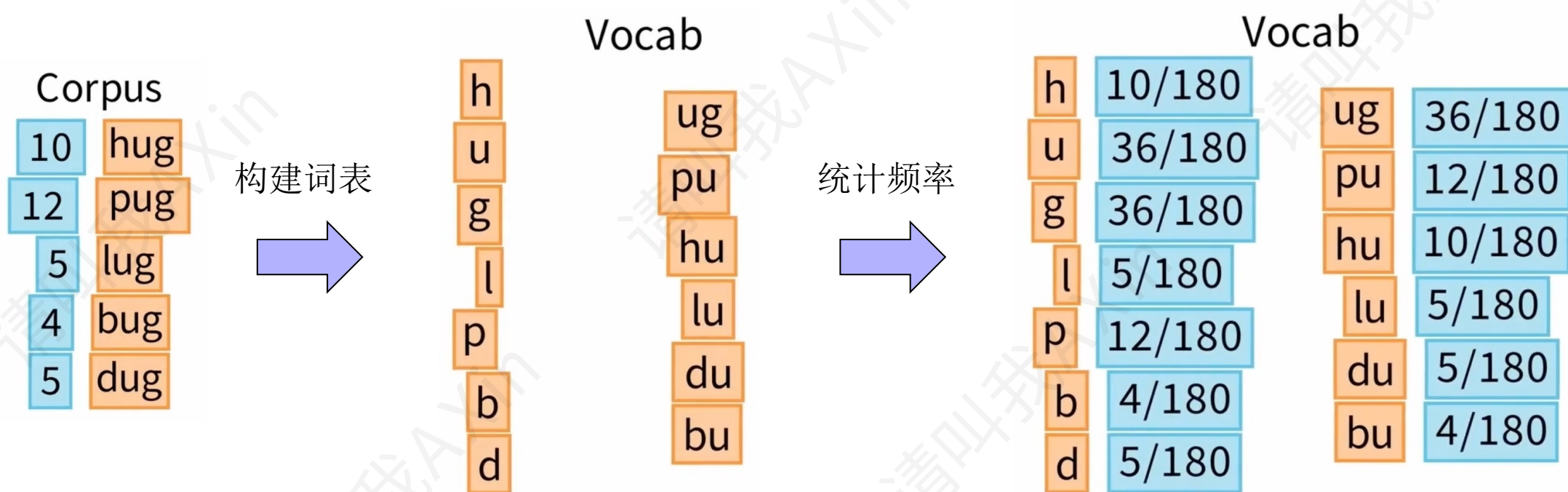
Unigram Tokenization

- Unigram model: 假设每个词的出现都是独立的



Unigram Tokenization

- 如何将文本切分成 token?



Unigram Tokenization

■ 如何将文本切分成 token?

根据词表计算每种切分的概率

Possible splits for "hug"

h	u	g	$\frac{10}{180} \times \frac{36}{180} \times \frac{36}{180} = 2.22e-03$
hu	g	$\frac{10}{180} \times \frac{36}{180} = 1.11e-02$	
h	ug	$\frac{10}{180} \times \frac{36}{180} = 1.11e-02$	
hug	0		

任选概率最大的切分方式

Vocab

h	10/180	ug	36/180
u	36/180	pu	12/180
g	36/180	hu	10/180
l	5/180	lu	5/180
p	12/180	du	5/180
b	4/180	bu	4/180
d	5/180		

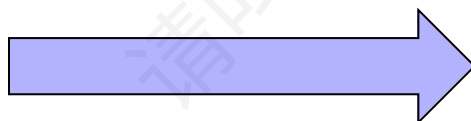
Corpus		Splits	Scores
10	hug	→ hu g	1.11e-02
12	pug		
5	lug		
4	bug		
5	dug		

Unigram Tokenization

- 如何将文本切分成 token?

Vocab			
h	10/180	ug	36/180
u	36/180	pu	12/180
g	36/180	hu	10/180
l	5/180	lu	5/180
p	12/180	du	5/180
b	4/180	bu	4/180
d	5/180		

根据词表对每个词进行划分



Corpus		Splits		Scores
10	hug	→	hu g	1.11e-02
12	pug	→	pu g	1.33e-02
5	lug	→	lu g	5.56e-03
4	bug	→	bu g	4.44e-03
5	dug	→	du g	5.56e-03

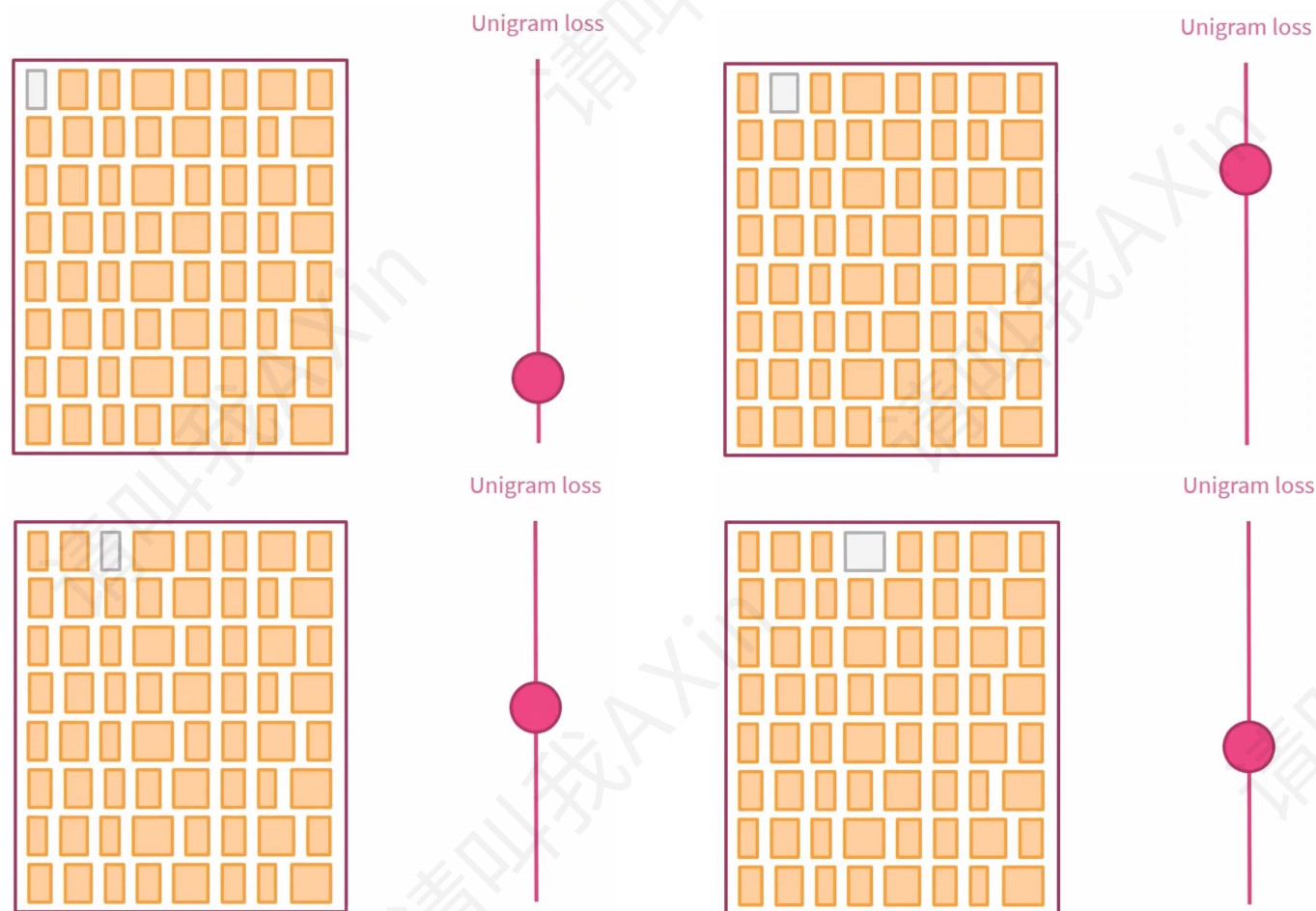
Unigram Tokenization

- 如何计算语料的 Unigram loss? 负对数似然

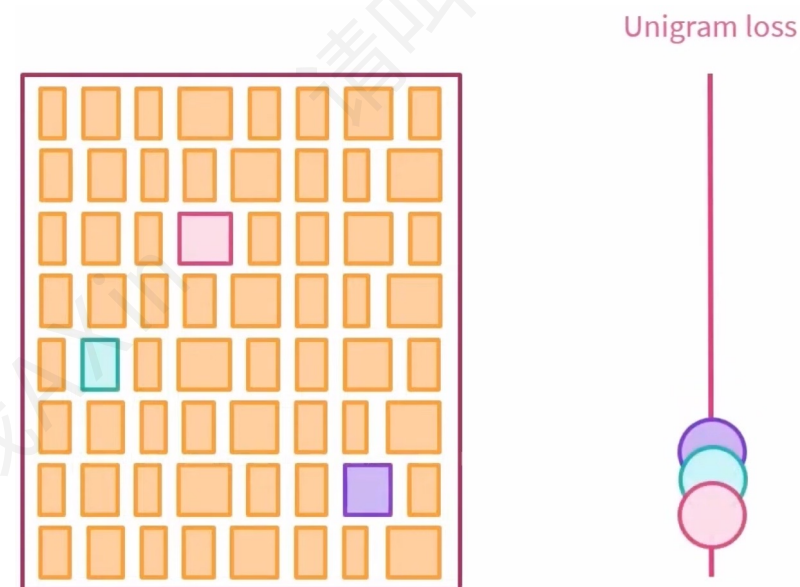
Corpus		Splits		Scores	Loss	
10	hug	→	hu g	1.11e-02	$\sum freq \times (-\log(P(word)))$	= 170.4
12	pug	→	pu g	1.33e-02	$10 \times (-\log(1.11e - 02))$	
5	lug	→	lu g	5.56e-03	$+12 \times (-\log(1.33e - 02))$	
4	bug	→	bu g	4.44e-03	$+5 \times (-\log(5.56e - 03))$	
5	dug	→	du g	5.56e-03	$+4 \times (-\log(4.44e - 03))$	
					$+5 \times (-\log(5.56e - 03))$	

Unigram Tokenization

■ 如何删减 token?



尝试删去一个 token，并计算对应的 unigram loss，删除 $p\%$ 使得 loss 增加最少的 token



注意：不删除基本token，如26个字母和标点

第1次迭代

Vocab

h	10/180	ug	36/180
u	36/180	pu	12/180
g	36/180	hu	10/180
l	5/180	lu	5/180
p	12/180	du	5/180
b	4/180	bu	4/180
d	5/180		

Possible splits for "hug"

h u g $\frac{10}{180} \times \frac{36}{180} \times \frac{36}{180} = 2.22e-03$

hu g $\frac{10}{180} \times \frac{36}{180} = 1.11e-02$

h ug $\frac{10}{180} \times 0 = 0.00e+00$

hug 0 = 0.00e+00

删掉ug

计算loss

Corpus		Splits	Scores
10	hug	→ hu g	1.11e-02
12	pug	→ pu g	1.33e-02
5	lug	→ lu g	5.56e-03
4	bug	→ bu g	4.44e-03
5	dug	→ du g	5.56e-03

loss无变化则随机删一个token

	Loss
With all vocabulary	170.4
Without ug	170.4
Without pu	170.4
Without hu	170.4
Without lu	170.4
Without du	170.4
Without bu	170.4

计算每种情况的loss

loss = 170.4

Corpus		Splits	Scores
10	hug	→ hu g	1.11e-02
12	pug	→ pu g	1.33e-02
5	lug	→ lu g	5.56e-03
4	bug	→ bu g	4.44e-03
5	dug	→ du g	5.56e-03

Loss

$$\sum freq \times (-\log(P(word)))$$
$$10 \times (-\log(1.11e-02)) + 12 \times (-\log(1.33e-02)) + 5 \times (-\log(5.56e-03)) + 4 \times (-\log(4.44e-03)) + 5 \times (-\log(5.56e-03))$$

注意：不删除基本token，如26个字母和标点

第1次迭代



注意：不删除基本token，如26个字母和标点

第2次迭代

Vocab

h	10/144
u	36/144
g	36/144
l	5/144
p	12/144
b	4/144
d	5/144

pu	12/144
hu	10/144
lu	5/144
du	5/144
bu	4/144

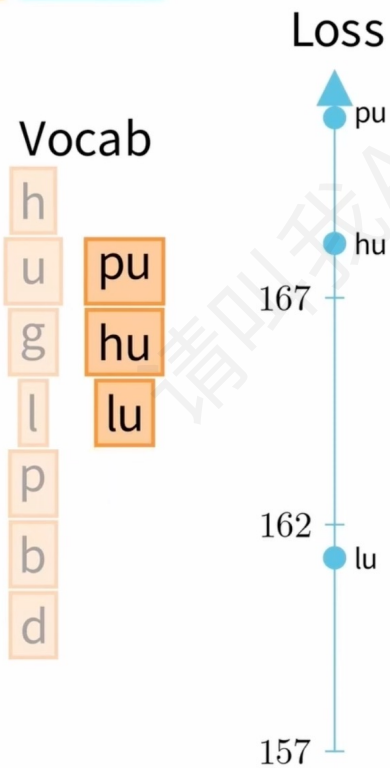
删掉hu

Possible splits for "hug"

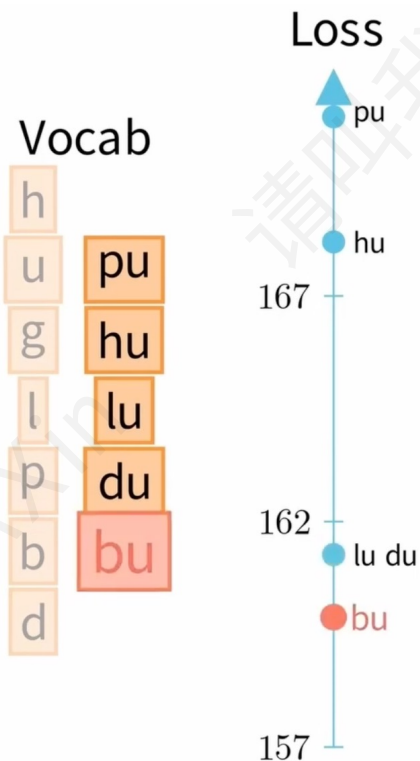
h	u	g	$\frac{10}{144} \times \frac{36}{144} \times \frac{36}{144} = 4.34e - 03$
hu	g	$0 \times \frac{36}{144} = 0.00e + 00$	
h	ug	$\frac{10}{144} \times 0 = 0.00e + 00$	
hug		$0 = 0.00e + 00$	

计算loss

Corpus	Splits	Scores
10 hug	→ h u g	4.34e-03
12 pug	→ pu g	2.08e-02
5 lug	→ lu g	8.68e-03
4 bug	→ bu g	6.94e-03
5 dug	→ du g	8.68e-03



删除40%的token



计算删除每个token之后的loss

$$\begin{aligned} \text{loss} = & 10 \times (-\log(4.34e-03)) \\ & + 12 \times (-\log(2.08e-02)) \\ & + 5 \times (-\log(8.68e-03)) \\ & + 4 \times (-\log(6.94e-03)) \\ & + 5 \times (-\log(8.68e-03)) = 168.2 \end{aligned}$$

Unigram Tokenization

- 迭代直到词表的大小缩减到预设值
 - Unigram 会遇到 unknown 的情况
- 为单词的每种划分计算概率非常耗时，采用更高效的**维特比 (Viterbi) 算法**

h	10/180	ug	36/180
u	36/180	pu	12/180
g	36/180	hu	10/180
l	5/180	lu	5/180
p	12/180	du	5/180
b	4/180	bu	4/180
d	5/180		

Possible splits for "hug"

h	u	g	$\frac{10}{180} \times \frac{36}{180} \times \frac{36}{180} = 2.22e-03$
hu	g	$\frac{10}{180} \times \frac{36}{180} = 1.11e-02$	
h	ug	$\frac{10}{180} \times \frac{36}{180} = 1.11e-02$	
hug		0	

Viterbi algorithm

SentencePiece Tokenization

<https://arxiv.org/pdf/1808.06226.pdf>

■ BPE、WordPiece、Unigram 的缺点

- 假设输入文本使用空格来分隔单词，但并非所有语言都使用空格来分隔单词（如中文、韩文、日文、阿拉伯语）
- 可以使用特定语言的 pre-tokenizer 分词，但不太通用

■ 为解决这个问题，SentencePiece

- 将输入视为输入字节流，包括空格
- 然后使用 Byte-level BPE 或 unigram 算法来构建适当的词汇表

■ Unigram 算法经常在 SentencePiece 中使用，是 ALBERT、T5、mBART、Big Bird 和 XLNet 等模型使用的 tokenization 算法

SentencePiece Tokenization

■ SentencePiece:

```
>>> from transformers import XLNetTokenizer
>>> tokenizer = XLNetTokenizer.from_pretrained("xlnet-base-cased")
>>> tokenizer.tokenize("Don't you love 🤗 Transformers? We sure do.")
['_Don', "'", 't', '_you', '_love', '_', '🤗', '_', 'Transform', 'ers', '?', '_We', '_sure', '_do', '.']
```

■ WordPiece:

```
>>> from transformers import BertTokenizer
>>> tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
>>> tokenizer.tokenize("I have a new GPU!")
['I', 'have', 'a', 'new', 'gp', '##u', '!']
```


Subword-based Tokenizers

暂停慢慢看

Tokenization method	Advantages	Disadvantages
BPE	Allows for a large vocabulary size Handles rare and out-of-vocabulary words well Efficiently tokenizes subwords	May result in many subwords for a single word Not ideal for languages without clear word boundaries
WordPiece	Allows for a large vocabulary size Handles rare and out-of-vocabulary words well Efficiently tokenizes subwords	May result in many subwords for a single word Not ideal for languages without clear word boundaries
Unigram	Efficiently tokenizes subwords Scales well for large vocabularies	May produce many subwords for a single word Not ideal for languages without clear word boundaries
SentencePiece with Unigram	Can handle languages without clear word boundaries Efficiently tokenizes subwords	May produce many subwords for a single word
SentencePiece with BPE	Can handle languages without clear word boundaries Efficiently tokenizes subwords	May result in many subwords for a single word
Byte-level BPE	Can handle any character without out-of-vocabulary tokens Efficiently tokenizes subwords	May result in many subwords for a single word
Word-level Tokenization	Simple to implement Fewer subwords per word than other methods	May not handle rare or out-of-vocabulary words well Not efficient for large vocabularies
Character-level Tokenization	Handles any word or character Efficient for small vocabularies	May produce too many subwords for a single word Not efficient for large vocabularies

总结

- Transformer 的输入是什么?
- Tokenizer 的作用
- 三种不同级别的 Tokenizers
 - Word-based
 - Character-based
 - Subword-based
- 四种 Subword-based Tokenizers
 - BPE/BBPE
 - WordPiece
 - Unigram
 - SentencePiece (使用 BBPE 或 Unigram)

Thanks!

请叫我AXin

2023 年 12 月 22 日